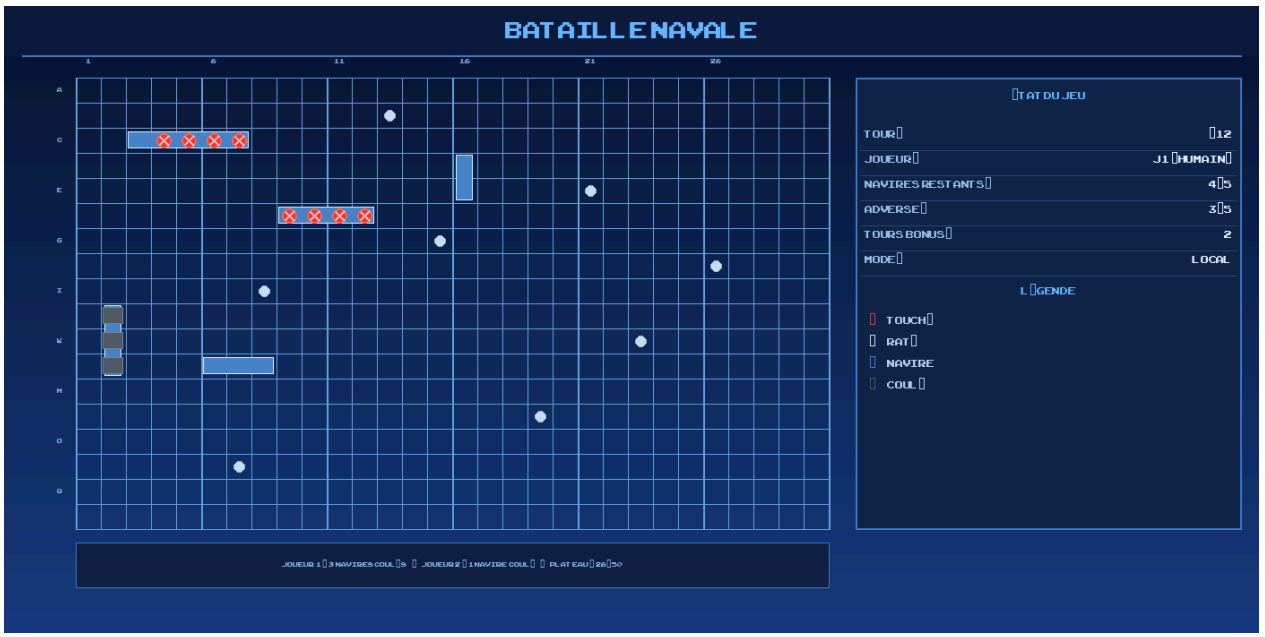
UNIVERSITÉ ÉVRY
PARIS-SACLAY

Université Évry Paris-Saclay — Département d'informatique
L3 MIAGE Formation Initiale — Année universitaire 2025-2026

RAPPORT DE PROJET

Bataille Navale



Membres du groupe

Aicha Tahar Bellar	20231526
Zakarya Zouhir	20232983
Adam Oualla Le Menec	20230456
Marwa Bourakkadi Idrissi	20226586

Encadrant : Guillaume Béduneau

Table des matières

Table des matières 2
1. Introduction 4
1.1 Présentation du projet 4
1.2 Contexte académique 4
1.3 Membres du groupe et affectation des tâches 4
2. Règles du jeu et spécifications 5
2.1 Le plateau de jeu 5
2.2 Composition de la flotte 5
2.3 Règles de placement 5
2.4 Actions disponibles par tour 6
2.5 Tours supplémentaires 6
2.6 Condition de victoire 6
3. Gestion de projet 6
3.1 Méthodologie : Cycle en V 6
3.2 Planning de réalisation 7
3.3 Affectation détaillée des tâches 7
3.4 Gestion des ressources 8
3.4.1 Outils utilisés 8
3.4.2 Architecture des modules Python 8
3.4.3 Collaboration Git 9
4. Modélisation UML 10
4.1 Diagramme de cas d'utilisation 10
4.2 Diagrammes d'activité 11
4.2.1 Configuration de la partie 11
4.2.2 Déroulement d'une partie 12
4.3 Diagrammes de séquence 13
4.3.1 Tir sur une case adverse 13
4.3.2 Règles de validité du plateau 15
4.3.3 Réception d'un tir — logique interne du Navire 17
4.3.4 Parcours d'une IA 19
4.4 Diagramme de classes 21
5. Développement du jeu 22
5.1 Architecture logicielle 22
5.2 Fonctionnalités implémentées 22
5.2.1 Fonctionnalités minimales 22
5.2.2 Fonctionnalités principales 22
5.2.3 Fonctionnalités bonus 22
5.3 Gestion du réseau local 23
5.4 Intelligence artificielle 23
5.4.1 IA facile — stratégie aléatoire 23
5.4.2 IA difficile — stratégie ciblée 23
6. Tests et validation 24
6.1 Stratégie de test 24
6.2 Cas de test principaux 24

7. Conclusion 25
7.1 Bilan du projet 25
7.2 Points forts 25
7.3 Pistes d'amélioration 25
7.4 Ce que nous avons retenu 25
Annexes 26
Annexe A — Structure du dépôt GitHub 26
Annexe B — Liste des diagrammes UML 26
Annexe C — Protocole réseau détaillé 27

1. Introduction

1.1 Présentation du projet

Ce projet a pour objectif de concevoir, modéliser et développer une version enrichie du jeu de société classique « Bataille Navale ». Le jeu doit permettre à deux joueurs de s'affronter en local ou via un réseau local, ou bien à un joueur humain de défier un adversaire virtuel (intelligence artificielle). Le développement respecte un cahier des charges précis définissant les règles du plateau, la disposition des navires et le déroulement des tours.

Le jeu se distingue du jeu original par plusieurs modifications notables : un grand plateau de 22 × 22 cases, une flotte à composition aléatoire mais équilibrée entre les deux camps, la possibilité de déplacer ses navires pendant la partie, et un système de tours supplémentaires accordés au joueur qui coule un navire adverse.

1.2 Contexte académique

Ce projet a été réalisé dans le cadre du cours de L3 MIAGE Formation Initiale à l'Université Évry Paris-Saclay, encadré par Guillaume Béduneau. Il s'inscrit dans une démarche pédagogique alliant modélisation orientée objet (UML), gestion de projet, travail collaboratif et développement logiciel.

Le projet s'est déroulé en deux grandes phases successives. La première (environ 6 semaines) a été consacrée à l'analyse des besoins et à la modélisation UML. La seconde (environ 8 semaines) a porté sur le développement du jeu en Python avec la bibliothèque Pygame.

1.3 Membres du groupe et affectation des tâches

Membre	N° étudiant	Responsabilité principale
Aicha Tahar Bellar	20231526	Gestion des tours, tours supplémentaires, détection fin de partie
Zakarya Zouhir	20232983	IA, coordination de la partie, mode réseau local
Adam Oualla Le Menec	20230456	Gestion du plateau, placement et validation des navires
Marwa Bourakkadi Idrissi	20226586	Tirs, déplacement des navires, affichage des messages

2. Règles du jeu et spécifications

2.1 Le plateau de jeu

Chaque joueur dispose d'un plateau de 22 × 22 cases. Les lignes sont identifiées par les lettres A à Z et les colonnes par des numéros de 1 à 22. Une case est désignée par une coordonnée de type « A1 », « B14 » ou « Z22 ».

Dans le sujet initial, il est demandé d'utiliser un plateau de **26 × 50 cases**. Cependant, après analyse et tests pratiques, nous avons décidé d'opter pour une grille de **22 × 22 cases**.

Plusieurs raisons motivent cette décision :

- **Lisibilité et ergonomie** :
  Une grille de 26 × 50 s'est révélée très grande à l'affichage, rendant l'interface difficile à lire et à manipuler pour le joueur.
- **Proportion avec les navires** :
  La taille maximale d'un navire étant de **5 cases**, une grille trop grande crée un déséquilibre :
  - les bateaux deviennent visuellement trop petits,
  - les interactions (tir, déplacement) deviennent moins précises.
- **Expérience utilisateur** :
  Une grille plus compacte (22 × 22) permet :
  - une meilleure visibilité globale du jeu,
  - une prise de décision plus rapide,
  - une expérience plus fluide.
- **Contraintes techniques** :
  L'affichage d'une grande grille nécessite soit :
  - de réduire fortement la taille des cases (peu lisable),
  - soit d'agrandir la fenêtre (peu pratique).

2.2 Composition de la flotte

Type de navire	Taille (cases)	Quantité par joueur
Porte-avions	5	1 à 2 (obligatoire)
Cuirassé	4	0 à 3
Sous-marin	3	0 à 3
Destroyer	3	0 à 3
Patrouilleur	2	0 à 3

L'attribution aléatoire respecte une contrainte d'équilibre : le nombre total de cases occupées par les deux flottes doit être globalement comparable.

2.3 Règles de placement

- Les navires peuvent être orientés horizontalement ou verticalement (pas en diagonale).
- Chaque navire doit être entièrement contenu dans les limites du plateau.
- Un espace minimum d'une case doit séparer chaque navire de ses voisins (toutes directions).
- Le placement est secret : chaque joueur positionne sa flotte sans que l'adversaire puisse la voir.

2.4 Actions disponibles par tour

- Tirer : annoncer les coordonnées d'une case adverse. Résultat immédiat : touché, coulé ou raté.
- Déplacer un navire : déplacer l'un de ses navires d'une case (N/S/E/O), sous réserve de validité.

2.5 Tours supplémentaires

- Chaque joueur commence son tour avec une action de base.
- Tant que le joueur possède au moins un porte-avions, chaque navire survivant (hors patrouilleurs et navires coulés) lui accorde un tour supplémentaire.
- Couler un navire adverse accorde immédiatement un tour supplémentaire.

2.6 Condition de victoire

La partie se termine lorsqu'un joueur a coulé l'intégralité des navires adverses. Ce joueur est déclaré vainqueur.

3. Gestion de projet

3.1 Méthodologie : Cycle en V

Nous avons suivi une méthodologie inspirée du cycle en V. Cette approche garantit que chaque phase de conception est validée avant de passer à la suivante, avec une correspondance directe entre les phases de conception (modélisation) et les phases de vérification (tests). Elle nous a permis d'avoir une vision globale et cohérente du système avant d'écrire la moindre ligne de code.

3.2 Planning de réalisation

Période	Phase	Activités principales
Semaine 1	Analyse et initialisation	Analyse du sujet, répartition des tâches, création GitHub, diagramme de cas d'utilisation
Semaines 2-3	Modélisation UML	Diagrammes de séquence, d'activité, de classes ; tableaux de spécifications ; corrections en TD
Semaines 4	Finalisation modélisation	Révision et cohérence des diagrammes, correction README, rapport phase 1
Semaines 5-7	Développement — noyau	Plateau, placement navires, logique tirs, déplacements, affichage Pygame
Semaines 8-12	Développement avancé	IA (facile/difficile), réseau local (sockets), sons, sprites, tours supplémentaires
Semaines 13-14	Tests et livraison	Tests unitaires, correction bugs, rapport final, journal Git, nettoyage dépôt

3.3 Affectation détaillée des tâches

Membre	Diagrammes UML	Développement
Adam Oualla Le Menec	Séquence « Validité plateau » ; Séquence « Parcours IA »	Gestion plateau 22×22, placement navires avec validation espacement, marquage cases
Aicha Tahar Bellar	Activité « Configuration de la partie »	Tours (base + bonus), suivi joueur courant, détection fin de partie, vainqueur
Zakarya Zouhir	Séquence « Réception d'un tir » ; Séquence « Tir adverse »	IA (stratégie aléatoire et ciblée), coordination partie, gestion réseau (sockets)
Marwa Bourakkadi Idrissi	Diagramme de classes ; Activité « Déroulement »	Tirs (touché/coulé/raté), déplacement navires, messages affichage (message.py)

3.4 Gestion des ressources

3.4.1 Outils utilisés

Outil	Catégorie	Usage dans le projet
Python 3.x	Langage	Langage principal de développement
Pygame	Bibliothèque	Affichage graphique, événements, sons
Socket (stdlib)	Bibliothèque	Communication réseau mode multijoueur LAN
Visual Studio Code	IDE	Environnement de développement, extensions Python et Git
Draw.io	Modélisation UML	Diagrammes activité, séquence, cas d'utilisation
LucidChart	Modélisation UML	Diagramme de classes, séquences complémentaires
Git / GitHub	Versioning	Gestion de versions, branches, pull requests
WhatsApp	Communication	Coordination quotidienne entre membres
Pygame — Sprites & Sons	Ressources	PNG navires, MP3 effets sonores, MID musiques, TTF polices

3.4.2 Architecture des modules Python

Le projet est structuré en packages Python distincts correspondant aux grandes responsabilités du système. Le tableau ci-dessous liste l'ensemble des fichiers sources avec leur rôle précis, tel qu'analysé depuis le dépôt livré.

Package / Fichier	Lignes (approx.)	Rôle précis
src/main.py	~600	Point d'entrée, menu principal, boucle Pygame principale, initialisation, Partie, constantes globales
src/jeuLocal.py	~250	Orchestration partie locale et réseau : run_network_game(), parse_response_result(), format_result_string()
src/reseauLocal.py	~90	Classe ReseauLocal : creer_partie(), rejoindre_partie(), envoyer(), recevoir(), ecouter()
src/joueurs/joueur.py	~55	Classes Joueur (abstraite) et JoueurHumain, enum ActionTour
src/joueurs/ia.py	~140	Classe JoueurVirtuel : logique IA facile/difficile, randomise_ships(), enemy_turn()
src/plateau/plateau.py	~200	Classe Plateau (22×22), grille, tirs, déplacements, validation espacement
src/plateau/case.py	~65	Classe Case, enums EtatCase et ResultatTir, méthode recevoirTir()
src/plateau/regles.py	~30	Règles de validation supplémentaires du plateau
src/navires/bateau.py	~90	Classe Bateau (sprite Pygame), orientations, encaisserTir(), calculerCasesApresDeplacement()
src/navires/deplacement.py	~40	Logique de déplacement des navires
src/affichage/message.py	~50	Fonctions utilitaires pour le rendu texte et panneaux d'interface Pygame

src/jeu/fin_partie.py	~40	Écran de fin de partie, annonce vainqueur, option rejeu

3.4.3 Collaboration Git

Le dépôt GitHub a constitué le point central de collaboration. Chaque membre a travaillé sur sa branche dédiée et soumis ses contributions via des pull requests. Le journal git livré comptabilise plus de 70 commits couvrant la durée totale du projet.

- Branches nommées explicitement par fonctionnalité (ex. : chichicha2006-sequence-tir, corrections_diagrammes, aicha).
- Messages de commits descriptifs (ex. : « Ajout de la V2 du diagramme recevoir un tir navire », « Refactor README.md for clarity and consistency »).
- Utilisation systématique des pull requests avec revue avant chaque fusion (10 PR documentées dans le journal).
- Journal Git maintenu à jour — plus de 70 commits sur la durée totale du projet.

Extrait représentatif du journal des commits :

```
2db424c  Merge pull request #10 — mise a jour rapport
5c9e526  Merge pull request #8 — modif_sequence_tir
9e4caa1  Ajout de la V2 du diagramme recevoir un tir navire
632139e  Merge pull request #6 — Refactor README.md
4d1b047  Update README with UML diagrams and tools
0150bef  ajout journal git
f7b5377  Initial commit
```

4. Modélisation UML

La phase de modélisation a produit huit diagrammes UML couvrant les visions fonctionnelle, dynamique et structurelle du système. Chaque diagramme est accompagné de son tableau de spécifications textuelles.

4.1 Diagramme de cas d'utilisation

Ce diagramme identifie les acteurs du système et les fonctionnalités disponibles. Deux acteurs principaux sont distingués : Joueur 1 (qui peut configurer la partie) et Joueur 2 / IA (qui rejoint et joue la partie avec les mêmes droits d'action).

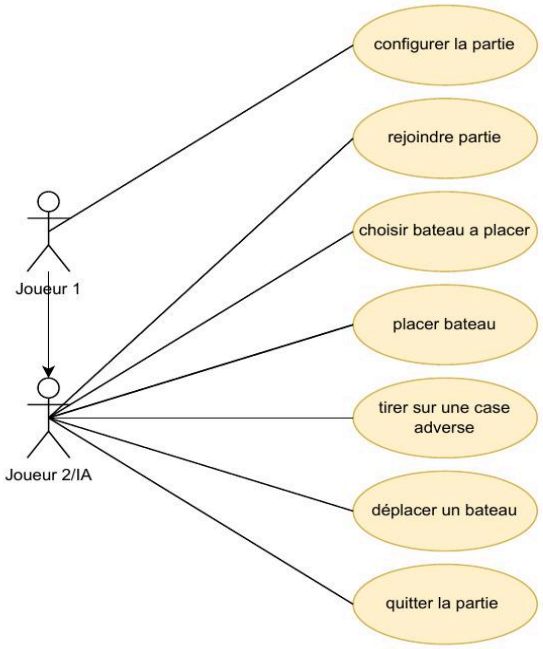


Figure 1 — Diagramme de cas d'utilisation

4.2 Diagrammes d'activité

4.2.1 Configuration de la partie

Ce diagramme décrit le processus de configuration initiale. Le joueur choisit le mode (local ou distant), puis l'adversaire (humain ou IA). En mode réseau, il peut créer ou rejoindre une partie existante. Une fois la configuration validée, les joueurs placent leurs navires à tour de rôle avant le début de la partie.

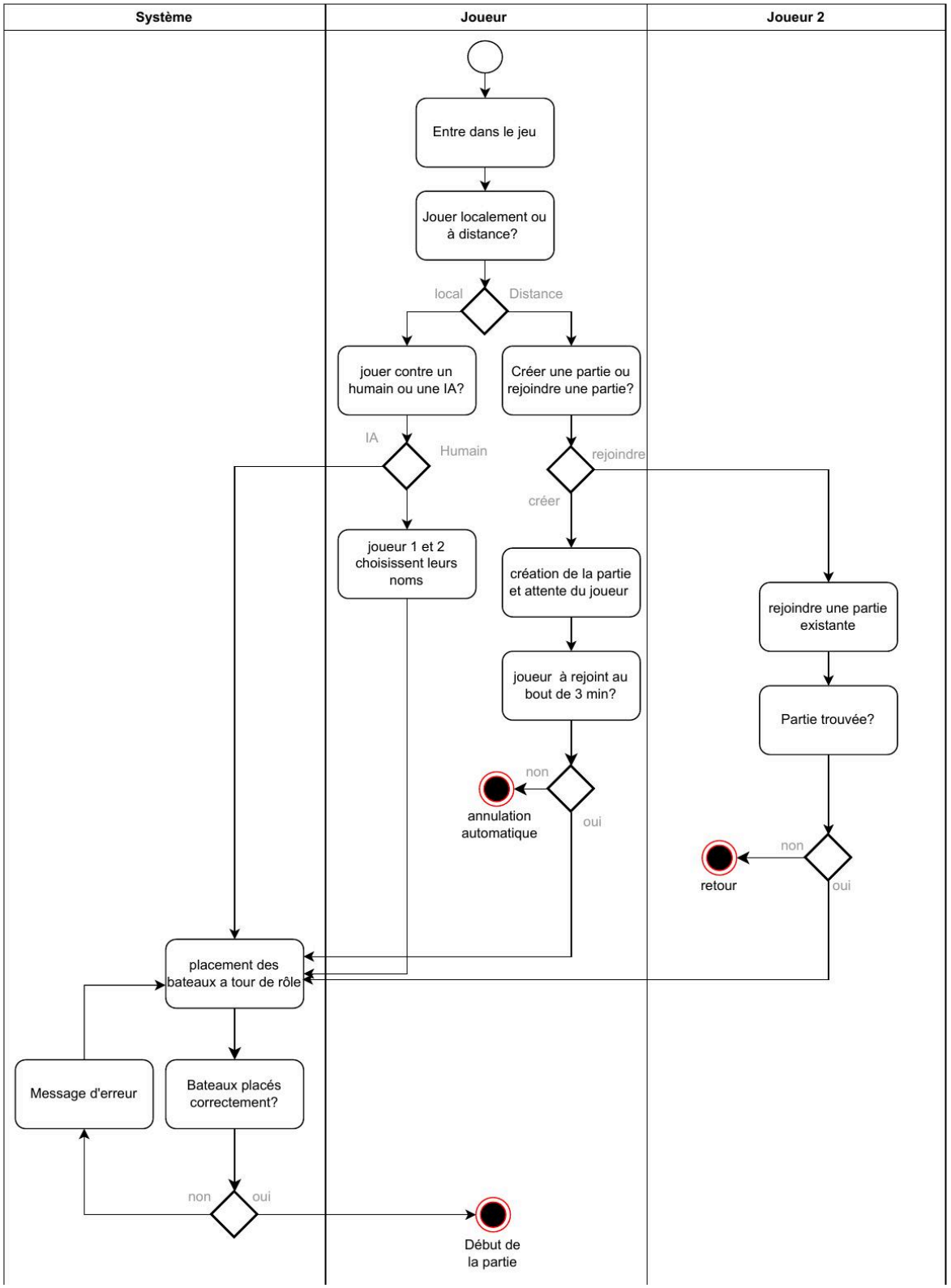


Figure 2 — Diagramme d'activité : Configuration de la partie

4.2.2 Déroulement d'une partie

Ce diagramme modélise le déroulement d'un tour (valable pour un joueur humain ou l'IA). À chaque tour, le joueur choisit entre tirer ou déplacer un navire. Le résultat du tir est traité et un tour supplémentaire est accordé si un navire est coulé.

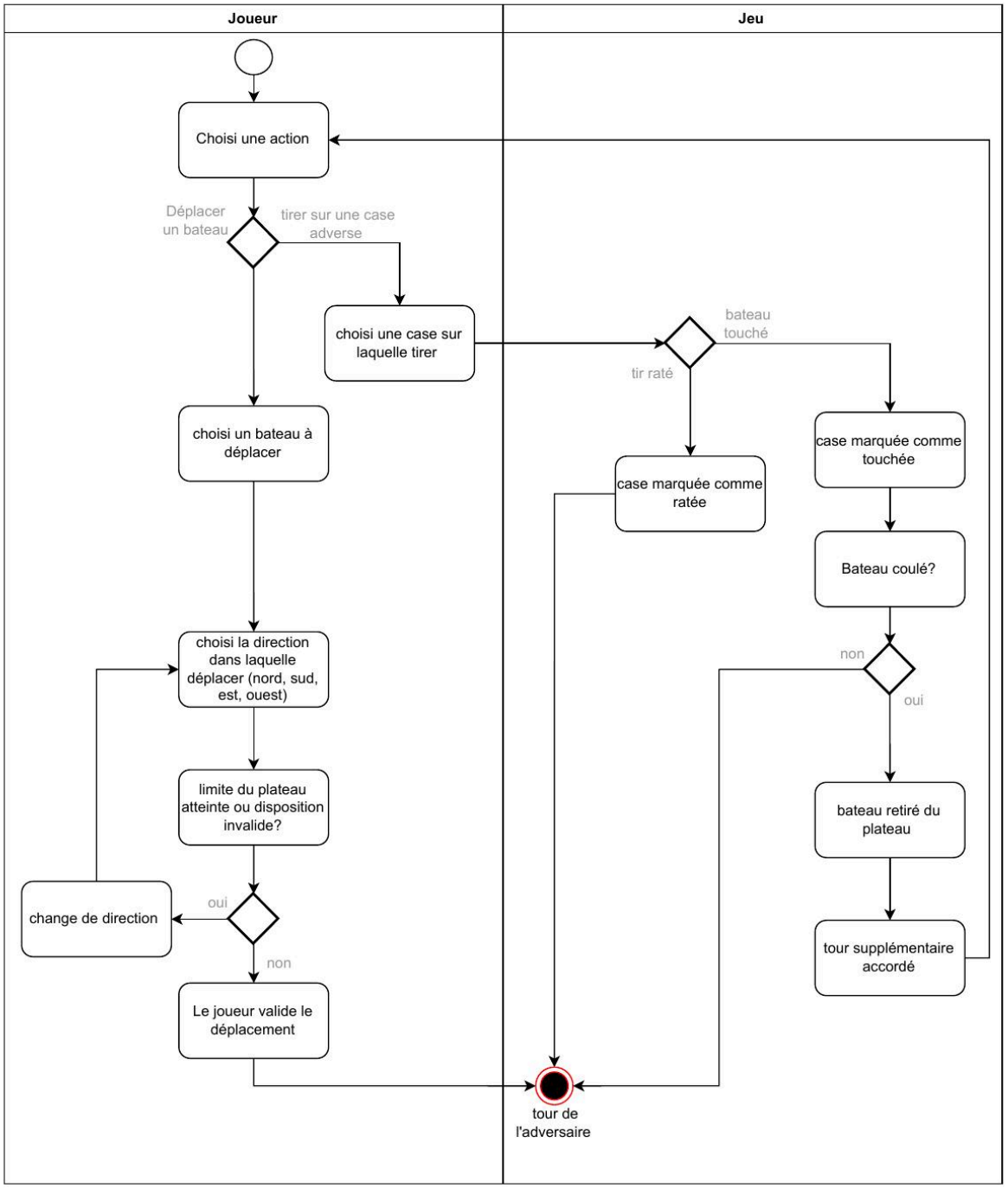


Figure 3 — Diagramme d'activité : Déroulement d'une partie

4.3 Diagrammes de séquence

4.3.1 Tir sur une case adverse

Ce diagramme décrit le processus complet d'un tir, de la sélection de la case jusqu'à l'affichage du résultat au joueur.

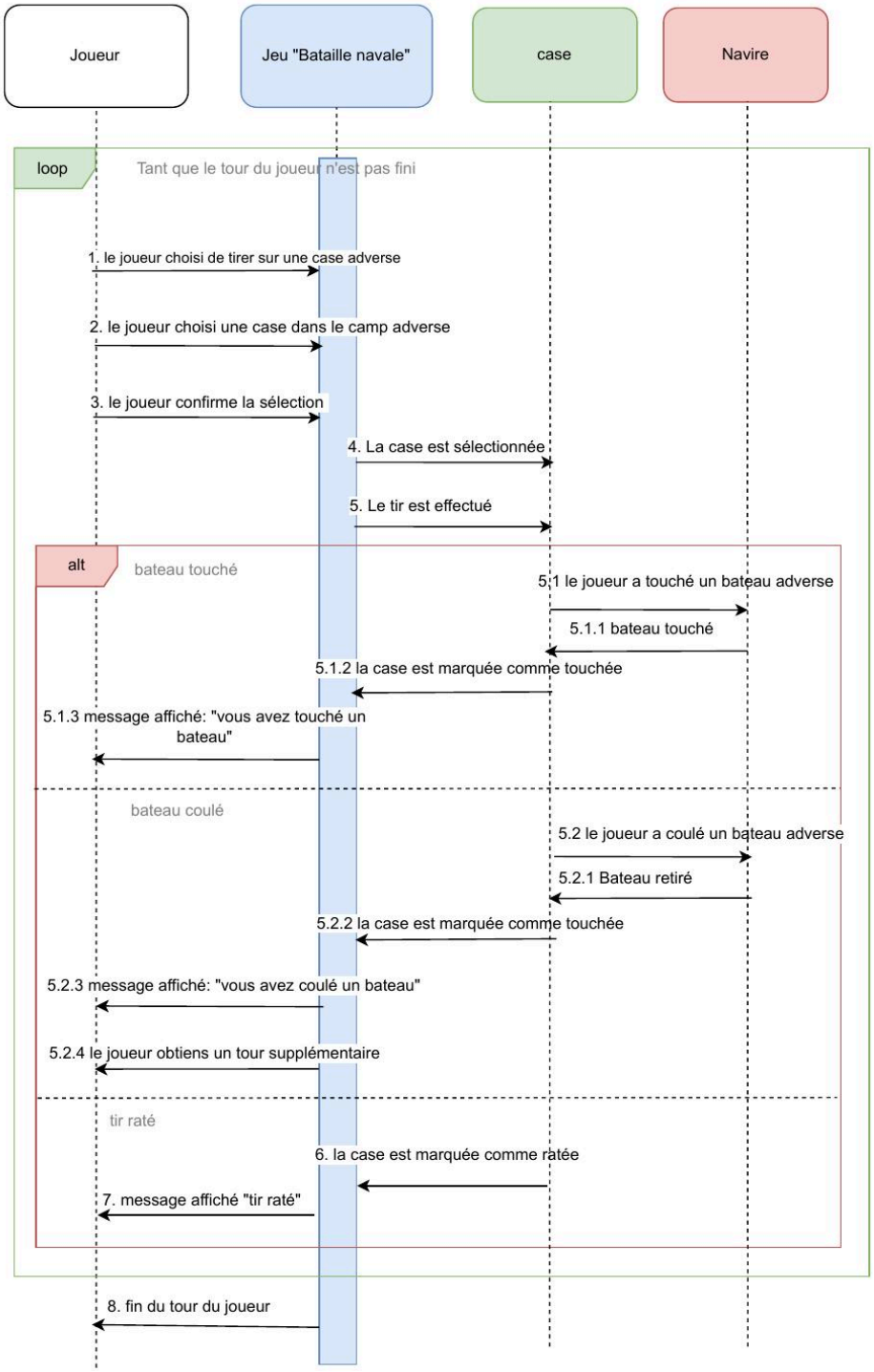


Figure 4 — Diagramme de séquence : Tir sur une case adverse

NOM	Tir sur une case adverse
ACTEURS	• Joueur • Jeu « Bataille navale » • Case • Navire
DESCRIPTION	Déroulement d'un tir lors d'une partie et affichage du résultat : touché, coulé ou raté.
SCÉNARIO NOMINAL	1. Le joueur choisit de tirer sur une case adverse. 2. Le joueur choisit une case dans le camp adverse. 3. Le joueur confirme la sélection. 4. La case est sélectionnée. 5. Le tir est effectué. 6. La case est marquée comme ratée. 7. Message affiché. 8. Fin du tour du joueur.
SCÉNARIOS ALTERNATIFS	5.1 Le joueur a touché un bateau adverse : 5.1.1 Bateau touché. 5.1.2 La case est marquée comme touchée. 5.1.3 Message affiché : « Vous avez touché un bateau ». 5.2 Le joueur a coulé un bateau : 5.2.1 Bateau retiré du plateau. 5.2.2 La case est marquée comme touchée. 5.2.3 Message affiché : « Vous avez coulé un bateau ». 5.2.4 Le joueur obtient un tour supplémentaire.
SCÉNARIOS D'EXCEPTIONS	E1 : La case sélectionnée a déjà été marquée (touchée, coulée, ratée).

4.3.2 Règles de validité du plateau

Ce diagramme détaille la validation du placement des navires : vérification des coordonnées, disponibilité de la case et respect de l'espacement minimum d'une case entre navires.

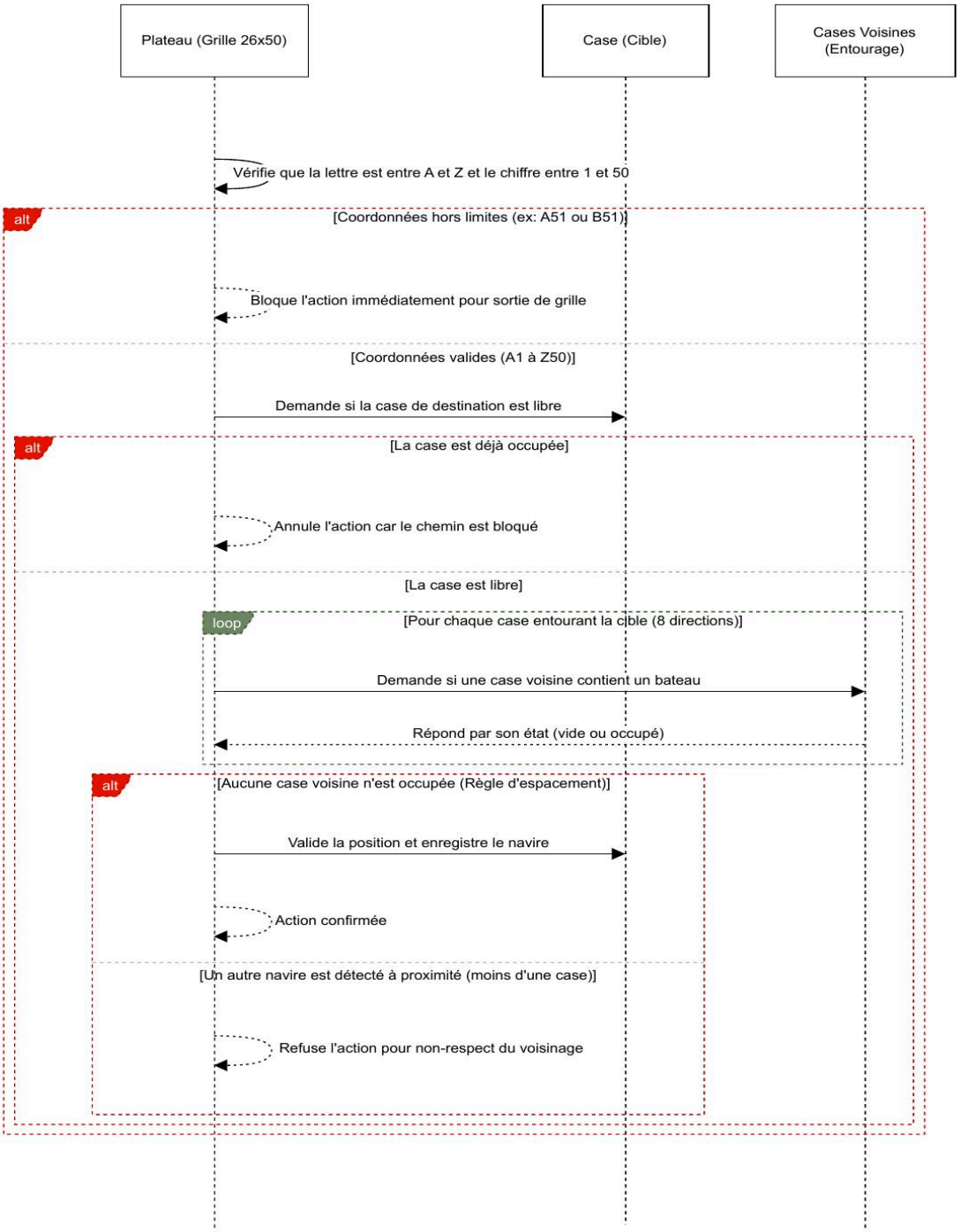


Figure 5 — Diagramme de séquence : Règles de validité du plateau

NOM	Règle de validité du plateau
ACTEURS	• Plateau • Case • Cases Voisines
DESCRIPTION	Le plateau valide le placement des navires en vérifiant auprès des cases qu'il n'y a aucune superposition ni contact avec le voisinage.
SCÉNARIO NOMINAL	1. Le plateau vérifie que la lettre est entre A et Z et le chiffre entre 1 et 50. 2. Le plateau constate que les coordonnées sont valides. 3. Le plateau demande à la case si elle est libre. 4. La case répond qu'elle est libre. 5. Le plateau demande si une case voisine contient un bateau. 6.1 La case voisine répond que son état est vide. 7. Le plateau valide la position et enregistre le navire.
SCÉNARIOS ALTERNATIFS	Il n'y a aucun scénario alternatif pour ce diagramme de séquence.
SCÉNARIOS D'EXCEPTIONS	Exception 1 — Coordonnées hors limites : 2.1 Les coordonnées sont hors limites. 2.2 Le plateau bloque l'action immédiatement pour sortie de grille. Exception 2 — Case déjà occupée : 4.1 La case répond qu'elle est déjà occupée. 4.2 Le plateau annule l'action car le chemin est bloqué. Exception 3 — Non-respect du voisinage : 6.1 Un autre navire est détecté à proximité (moins d'une case). 6.2 Le plateau refuse l'action pour non-respect du voisinage.

4.3.3 Réception d'un tir — logique interne du Navire

Ce diagramme modélise la méthode recevoirTir(x,y) de la classe Navire : vérification anti-doublon, enregistrement du touché, décrémentation des PV et retour du résultat (TOUCHÉ, COULÉ ou RATÉ).

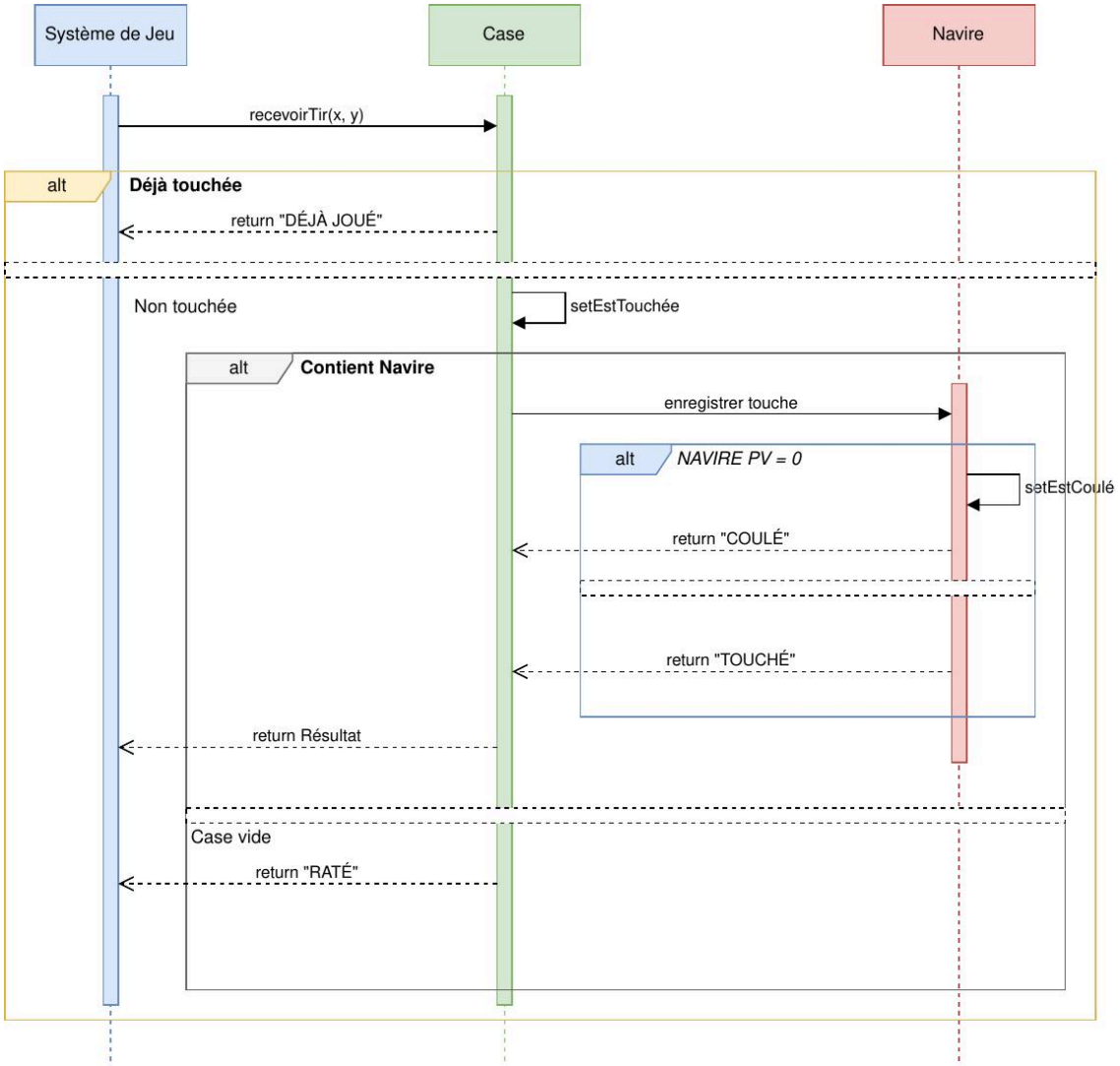


Figure 6 — Diagramme de séquence : Réception d'un tir (logique interne du Navire)

NOM	Navire Recevoir Tir
ACTEURS	• Système de Jeu • Case • Navire
DESCRIPTION	Le système de jeu transmet les coordonnées d'un tir à une case spécifique pour déterminer le résultat (Touché, Coulé ou Raté) et mettre à jour l'état du plateau.
SCÉNARIO NOMINAL	1. Le Système de Jeu envoie la commande recevoirTir à la Case. 2. La Case vérifie son état (elle n'a pas encore été jouée). 3. La Case change son état interne : setEstTouchée. 4. La Case détecte qu'elle contient une référence vers un Navire. 5. La Case demande au Navire d'encaisser les dégâts : enregistrerTouche(). 6. Le Navire décrémente ses points de vie. 7. Le Navire vérifie qu'il lui reste des points de vie (PV > 0). 8. Le Navire renvoie le message « TOUCHÉ » à la Case. 9. La Case renvoie le résultat final « TOUCHÉ » au Système de Jeu.
SCÉNARIOS ALTERNATIFS	Alternatif 1 — Le navire est coulé : 7.1 Le Navire constate que ses points de vie sont à 0. 7.2 Le Navire change son état : setEstCoulé. 7.3 Le Navire renvoie le message « COULÉ ». Alternatif 2 — Tir dans l'eau (raté) : 4.1 La Case constate qu'elle est vide (pas de navire). 4.2 La Case renvoie immédiatement « RATÉ » au Système de Jeu.
SCÉNARIOS D'EXCEPTIONS	Exception 1 — Case déjà jouée : 2.1 La Case détecte qu'elle a déjà été visée précédemment (estTouchée = true). 2.2 La Case renvoie « DÉJÀ JOUÉ » sans modifier l'état du navire.

4.3.4 Parcours d'une IA

Ce diagramme décrit le comportement de l'IA lors d'un tour : consultation de l'état de la partie, choix d'une action (tirer ou déplacer), exécution et gestion des tours bonus en cas de coulage.

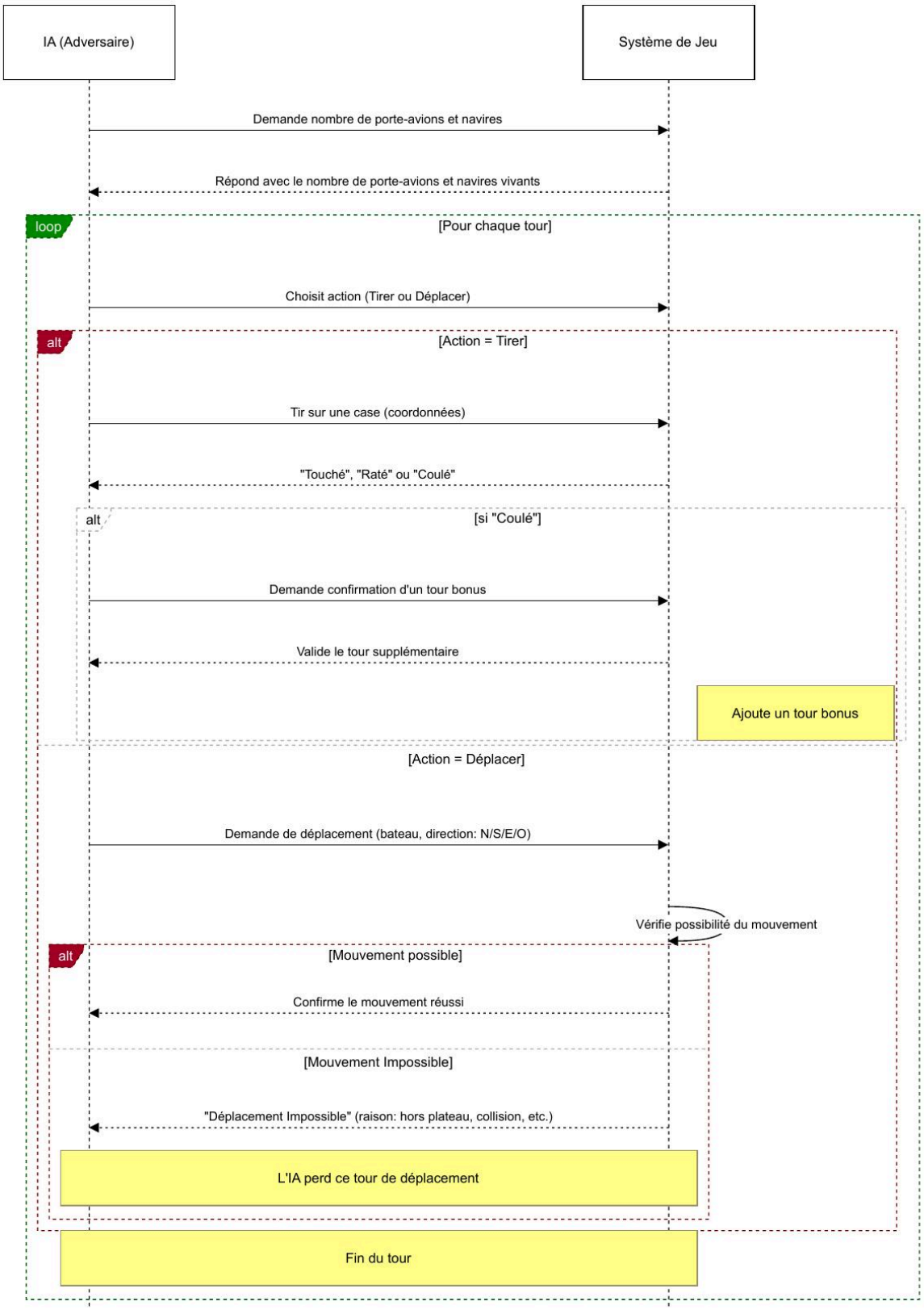


Figure 7 — Diagramme de séquence : Parcours d'une IA

NOM	Déroulement d'un tour de l'IA (tir ou déplacement)
ACTEURS	• IA (Adversaire) • Système de jeu
DESCRIPTION	L'IA interagit avec le système de jeu pour connaître l'état des navires, choisir une action (tirer ou déplacer) et l'exécuter selon les règles définies.
SCÉNARIO NOMINAL	1. L'IA demande au système le nombre de porte-avions et de navires encore vivants. 2. Le système répond avec le nombre de porte-avions et de navires vivants. 3. Pour le tour en cours, l'IA choisit une action : Tirer ou Déplacer. Cas 1 — Action = Tirer : 4. L'IA tire sur une case en envoyant les coordonnées. 5. Le système renvoie le résultat : « Touché », « Raté » ou « Coulé ». Cas 2 — Action = Déplacer : 6. L'IA demande un déplacement (bateau, direction N/S/E/O). 7. Le système vérifie la possibilité du mouvement. 8. Le système confirme que le mouvement est réussi.
SCÉNARIOS ALTERNATIFS	Résultat tir = « Coulé » : 5.1 L'IA demande la confirmation d'un tour bonus. 5.2 Le système valide le tour supplémentaire. 5.3 Un tour bonus est ajouté.
SCÉNARIOS D'EXCEPTIONS	8.1 Le système indique « Déplacement impossible » (hors plateau, collision, etc.). 8.2 L'IA perd ce tour de déplacement.

4.4 Diagramme de classes

Le diagramme de classes constitue la colonne vertébrale de l'architecture du système. Il décrit les classes principales (Partie, Joueur, Plateau, Navire, Case), leurs attributs, leurs méthodes et les relations qui les lient.

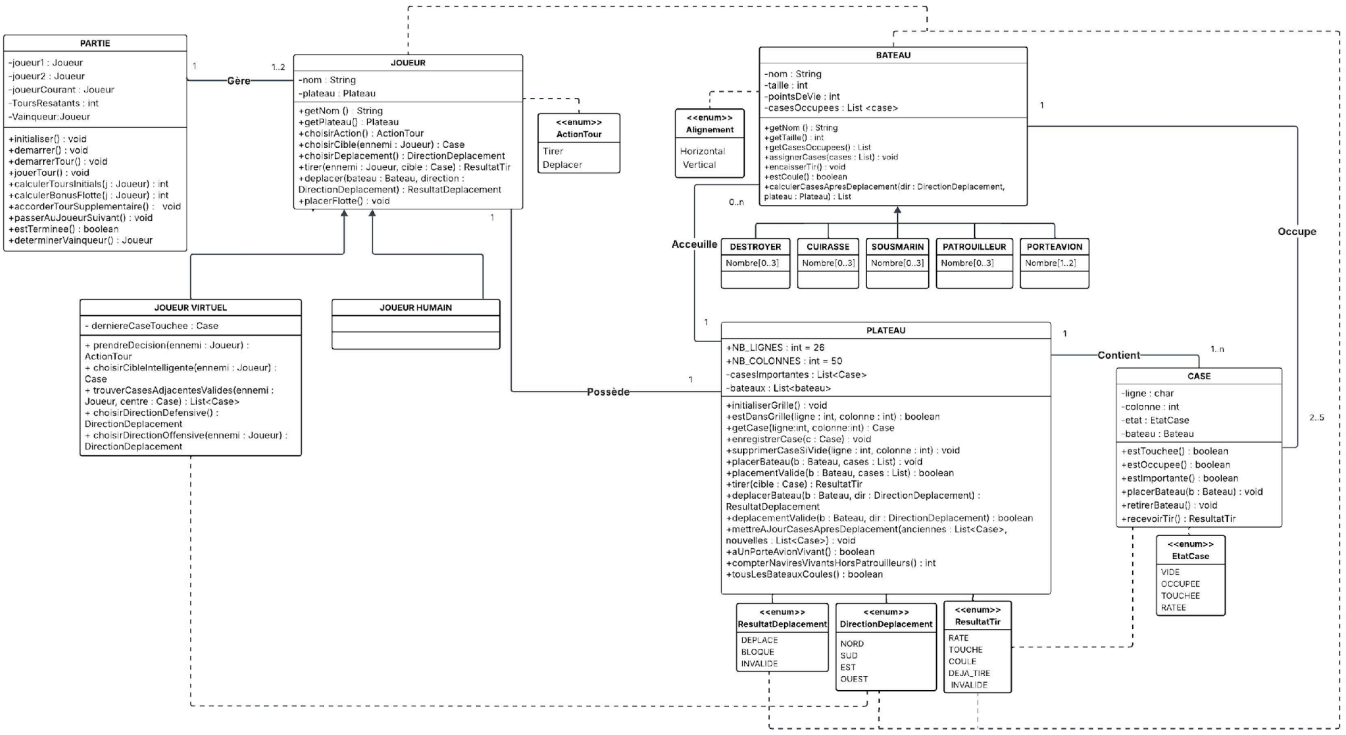


Figure 8 — Diagramme de classes

Les classes principales identifiées sont :

- Partie : gère le déroulement global, les deux joueurs, les tours et la détection de fin de partie.
- Joueur (abstrait) : définit les actions communes ; spécialisé en JoueurHumain et JoueurVirtuel.
- JoueurVirtuel : hérite de Joueur et implémente la logique de l'IA.
- Plateau : grille 26×50, valide les tirs et déplacements, maintient l'état des cases.
- Navire (abstrait) : taille, cases occupées, PV ; spécialisé en PorteAvions, Cuirassé, SousMarin, Destroyer, Patrouilleur.
- Case : état (VIDE, OCCUPEE, TOUCHEE, RATEE), référence optionnelle vers un Navire.

5. Développement du jeu

5.1 Architecture logicielle

Module	Catégorie	Rôle
main.py	Point d'entrée	Menu principal, initialisation Pygame, lancement du mode choisi
jeuLocal.py	Jeu local	Orchestration partie locale (2 joueurs ou joueur vs IA)
reseauLocal.py	Réseau	Gestion sockets TCP pour mode multijoueur LAN
fin_partie.py	Fin de partie	Écran de fin, annonce vainqueur, option rejeu
affichage/message.py	Affichage	Fonctions utilitaires rendu texte, panneaux d'interface

5.2 Fonctionnalités implémentées

5.2.1 Fonctionnalités minimales

- Plateau 22x22 avec coordonnées lettre-chiffre.
- Placement aléatoire et équilibré des navires avec respect des règles d'espacement.
- Système de tir avec retour immédiat (touché, coulé, raté).
- Système de déplacement des navires avec validation de la direction.
- Détection automatique de fin de partie et annonce du vainqueur.

5.2.2 Fonctionnalités principales

- Mode joueur vs joueur en local (confidentialité du plateau assurée par un écran de transition).
- Mode joueur vs IA avec deux niveaux : IA facile (tirs aléatoires) et IA difficile (stratégie de ciblage).
- Système de tours supplémentaires complet (porte-avions actif + coulage d'un navire).
- Interface graphique avec Pygame (grille, navires, marqueurs de tir hit/miss).

5.2.3 Fonctionnalités bonus

- Mode multijoueur en réseau local (sockets Python, architecture client-serveur).
- Effets sonores : boom1.mp3, boom2.mp3, boom3.mp3 (tirs), splash1-3.mp3 (rate), sink.mp3 (coulé).
- Musiques d'ambiance : mars.mid, valkyries.mid.
- Sprites graphiques : Battleship5.png, Cruiser4.png, Destroyer2.png, Submarine3.png, RescueShip3.png.
- Polices thématiques : ARCADECLASSIC.TTF, INVASION2000.TTF.
- Marqueurs visuels : hit.png et miss.png.

5.3 Gestion du réseau local

Le module reseauLocal.py implémente la communication entre deux machines en réseau local. L'architecture est client-serveur :

- Serveur (créateur de partie) : initialise un socket TCP, écoute sur un port défini, attend la connexion.
- Client (rejoindre une partie) : se connecte à l'adresse IP et au port du serveur.
- Échanges : actions de jeu (coordonnées de tir, direction de déplacement) et confirmations de résultat.
- Confidentialité : chaque joueur ne transmet que ses actions, jamais l'état de son propre plateau.

5.4 Intelligence artificielle

5.4.1 IA facile — stratégie aléatoire

L'IA facile sélectionne aléatoirement une case non encore ciblée parmi toutes les cases disponibles. Pour le déplacement, elle choisit aléatoirement un navire et une direction valide.

5.4.2 IA difficile — stratégie ciblée

L'IA difficile adopte une stratégie en deux phases : en mode « chasse » (aucun navire touché récemment), elle tire en suivant un pattern de damier pour maximiser la couverture ; en mode « destruction » (navire touché sans être coulé), elle explore systématiquement les cases adjacentes pour le localiser et le couler.

6. Tests et validation

6.1 Stratégie de test

- **Tests unitaires** : vérification des méthodes isolées (validation placement, calcul tours bonus, détection fin de partie).
- **Tests d'intégration** : interactions entre modules (plateau + navires, jeu + IA, jeu + réseau).
- **Tests fonctionnels** : parties complètes jouées manuellement pour valider le respect du cahier des charges.

6.2 Cas de test principaux

Cas de test	Résultat attendu	Statut
Placement hors limites du plateau	Refus du placement	✓ Validé
Placement à moins d'une case d'un autre navire	Refus du placement	✓ Validé
Tir sur une case déjà jouée	Retour DEJA_TIRE sans modification	✓ Validé
Coulage du dernier navire adverse	Fin de partie + vainqueur déclaré	✓ Validé
Déplacement vers une case hors limites	Déplacement refusé	✓ Validé
Tours supplémentaires après coulage	Tour bonus accordé correctement	✓ Validé
Connexion réseau LAN (client-serveur)	Connexion établie, actions transmises	✓ Validé
Envoi TIR en mode réseau	RESULT reçu et grille mise à jour	✓ Validé
GAME_OVER réseau (dernier bateau coulé)	Écran de victoire/défaite affiché	✓ Validé
Placement IA (randomise_ships)	Flotte valide placée en < 3000 essais/navire	✓ Validé

7. Conclusion

7.1 Bilan du projet

Ce projet nous a permis de mener un cycle complet de développement logiciel, de l'analyse des besoins jusqu'à la livraison d'un produit fonctionnel. La phase de modélisation nous a appris à formaliser des règles complexes sous forme de diagrammes UML cohérents, anticipant les cas d'usage et d'exception avant d'écrire le code. La phase de développement nous a confrontés aux défis concrets de l'implémentation : gestion de l'état du jeu, synchronisation réseau, cohérence de l'interface graphique.

7.2 Points forts

- Couverture complète des fonctionnalités du sujet, incluant toutes les fonctionnalités bonus.
- Organisation collaborative rigoureuse grâce à Git (branches, pull requests, journal de bord, plus de 70 commits).
- Architecture modulaire en packages Python distincts facilitant la maintenance et l'évolution du code.
- Implémentation réseau fonctionnelle avec protocole textuel clair, mode non-bloquant et gestion des erreurs.
- Diagrammes UML complets et cohérents, validés collectivement en séance de TD.
- IA difficile dotée d'une heuristique de chasse/destruction à deux phases bien structurée.

7.3 Pistes d'amélioration

- IA difficile : heuristiques plus avancées (damier en phase chasse, backtracking en phase destruction).
- Mode réseau : transmission des déplacements de navires à l'adversaire (message MOVE <navire> <direction>).
- Mode réseau : gestion de la déconnexion en cours de partie et reconnexion.
- Extension du mode réseau à Internet (traversée NAT, WebSocket ou serveur relais).
- Interface de placement des navires plus ergonomique (glisser-déposer).
- Sauvegarde de partie permettant de reprendre une partie interrompue.
- Mode tournoi (plusieurs joueurs successifs).

7.4 Ce que nous avons retenu

Au-delà des compétences techniques acquises, ce projet nous a confirmé l'importance de la communication et de l'organisation dans le travail d'équipe. La répartition claire des tâches dès le départ, couplée à des outils adaptés (GitHub, WhatsApp), a assuré un déroulement fluide. La rigueur apportée à la modélisation s'est révélée un investissement rentable : les diagrammes UML ont constitué une référence solide tout au long du développement. L'implémentation du mode réseau et de l'IA nous a par ailleurs permis d'approfondir des notions de programmation concurrente et d'algorithmique que nous n'avions qu'effleurées en cours.

Annexes

Annexe A — Structure du dépôt GitHub

Répertoire / Fichier	Contenu
/	README.md, journal_git.txt, .gitignore, .vscode/
/Diagrammes/	8 diagrammes UML au format PDF (Draw.io / LucidChart)
/src/	main.py, jeuLocal.py, reseauLocal.py, sources.txt
/src/affichage/	message.py (module affichage)
/src/jeu/	fin_partie.py (écran de fin)
/src/joueurs/	joueur.py, ia.py
/src/plateau/	plateau.py, case.py, regles.py
/src/navires/	bateau.py, deplacement.py
/src/Fonts/	ARCADECLASSIC.TTF, INVASION2000.TTF
/src/Sounds/	boom1-3.mp3, splash1-3.mp3, sink.mp3, mars.mid, valkyries.mid
/src/Sprites/	Battleship5.png, Cruiser4.png, Destroyer2.png, Submarine3.png, RescueShip3.png, hit.png, miss.png

Annexe B — Liste des diagrammes UML

Diagramme	Type	Fichier PDF
Cas d'utilisation	Fonctionnel	diagramme_cas_utilisation.drawio.pdf
Activité — Configuration	Dynamique	activité_configuration_jeu.drawio.pdf
Activité — Déroulement	Dynamique	activité_déroulementDePartie.drawio.pdf
Séquence — Tir adverse	Dynamique	séquence_tirer_case_adverse.drawio.pdf
Séquence — Validité plateau	Dynamique	séquence_cas_Plateau_Cas.drawio.pdf
Séquence — Réception tir	Dynamique	séquence_recevoire_tir_navire.drawioV2.pdf
Séquence — Parcours IA	Dynamique	séquence_cas_IA.drawio.pdf
Diagramme de classes	Structurel	diagrammeDeClassee.pdf

Annexe C — Protocole réseau détaillé

Tableau complet des messages du protocole ReseauLocal :

Message	Direction	Déclencheur	Action côté récepteur
READY	Bidirectionnel	Fin du placement des navires	Synchronisation : la partie commence
TIR <l> <c>	Joueur actif → passif	Clic sur grille ennemie	recevoirTir() sur le plateau local, envoi de RESULT
RESULT TOUCHE <l> <c>	Passif → actif	Navire touché	Marque case TOUCHÉE, affiche hit.png, fin du tour actif
RESULT COULE <l> <c>	Passif → actif	Navire coulé	Marque case TOUCHÉE, vérifie GAME_OVER
RESULT RATE <l> <c>	Passif → actif	Tir dans l'eau	Marque case RATÉE, affiche miss.png, fin du tour actif
RESULT INVALIDE <l> <c>	Passif → actif	Coordonnées hors grille	Message d'erreur affiché
GAME_OVER WIN	Passif → actif	Tous les bateaux du passif coulés	Écran « Victoire ! » côté actif